

OFFICIAL ACADEMIC CURRICULUM SPECIFICATION

2-Year Professional Software Engineering & Full-Stack Diploma

Diploma in Software Development & Cloud Systems

Duration: 2 Years (4 Semesters)
Total Effort: ~1,200 Practical Hours
Generated: August 16, 2026

Sem 1: Foundations & Modern Web

Sem 2: Frontend, Backend & SQL

Sem 3: Next.js, Cloud & Docker

Sem 4: Mobile, AI & Capstone

🎓 2-YEAR COMPREHENSIVE SOFTWARE DEVELOPER CURRICULUM

Program Philosophy & Learning Architecture

Designed to take an absolute beginner from foundational computing, web development, and browser DevTools through to professional full-stack development, cloud deployment, cross-platform mobile apps (Flutter), and applied generative AI systems.

KEY PEDAGOGICAL HIGHLIGHTS:

- ✔ Project-Driven Hands-On Coding (70% Practical / 30% Theory)
- ✔ Daily Git Commits & Industry Code Reviews
- ✔ Real-World Developer Tooling (DevTools, Docker, Postman, CI/CD)
- ✔ Weekly Milestone Submissions & 1-on-1 Mentorship
- ✔ Production-Grade Capstone Deployments

GRADUATE CAREER PROFILES:

Full-Stack Software Engineer

Frontend Specialist (React/Next.js)

Backend & API Engineer (Node.js/PostgreSQL)

Cross-Platform Mobile Developer (Flutter/Android)

Cloud & DevOps Associate

Prerequisites: Basic computer literacy and logical problem-solving aptitude. No prior coding experience required.



OFFICIAL CURRICULUM

2-YEAR PROGRAM • 4 SEMESTERS

ZERO TO INDUSTRY READY

2-Year Professional Software Engineering & Full-Stack Diploma

Diploma in Software Development & Cloud Systems • Comprehensive 96-Week Full-Stack, Mobile & AI Engineering Syllabus



Duration
4 Semesters
(24 Mo.)



Total Effort
~1,200
Practical Hours



Capstone Projects
7 Production
Builds



Outcome
Job-Ready
Engineer

SEMESTER 1 • BEGINNER FOUNDATIONS

Months 1–6 • Weeks 1–24 • ~300 Practical Hours

Foundations of Computing & Modern Web Development

From Zero to Building Responsive Interactive Web Experiences

Establishes rock-solid computing fundamentals, modern web standards, developer workflow mastery (DevTools, Git, Terminal), and core algorithmic programming using JavaScript.



SEMESTER CORE GOAL: Students build, inspect, debug, and publish fully responsive, accessible, and interactive web applications using semantic HTML5, modern CSS3, vanilla JavaScript, and Git/GitHub.

6 Modules

2 Capstone Projects

WEB CORE

HTML5

CSS3

JavaScript (ES6)

DOM API

Web Storage

DEVELOPER TOOLS

Chrome DevTools

Inspect Element

Lighthouse

VS Code

Terminal/CLI

VERSION CONTROL

Git

GitHub

GitLab

Markdown

Git Flow

DESIGN & UX

Responsive Design

Flexbox

CSS Grid

Figma to HTML

Accessibility (a11y)

Detailed Course Modules & Units

Showing 6 of 6 Modules

Computing Architecture, Internet Protocols & Developer Environment

CS101

Weeks 1–3 (36 Hours)

Understand how computers, operating systems, networks, browsers, and the...

CURRICULUM TOPICS & PRACTICAL LABS

How the Internet Works & Client-Server Architecture

Tools: Terminal cURL DNS lookup tools Brave/Chrome

Understanding IP addresses, DNS resolution, TCP/IP packets, HTTP/HTTPS request-response cycles, and Web Server mechanics.

KEY CONCEPTS: DNS resolution pipeline HTTP status codes & headers

Client-Server model URL anatomy & ports

 **Hands-on Lab Exercise:** Network tracing using Wireshark & traceroute; configuring local localhost web servers.

Developer Terminal & Unix Command-Line Mastery

Tools: Bash Zsh VS Code Integrated Terminal

Navigating file systems, process management, file permissions, shell scripting basics, and environment variables.

KEY CONCEPTS: Bash/Zsh navigation Piping & redirection

File manipulation (grep, sed, awk) SSH basics

 **Hands-on Lab Exercise:** Building a shell script to automate directory backups and project scaffolding.

VS Code & Development Ecosystem Configuration

Tools: VS Code Live Server GitLens ESLint Prettier

Mastering code editor ergonomics, essential extensions, linters (Prettier, ESLint), keyboard shortcuts, and workspace settings.

KEY CONCEPTS: Workspace configuration Prettier & format on save

Snippet generation

Emmet shortcuts



Hands-on Lab Exercise: Customizing a professional VS Code environment optimized for frontend web workflows.



STUDENT COMPETENCIES & MEASURABLE OUTCOMES

- Explain the end-to-end lifecycle of a web request from browser to server and back.
- Set up and maintain a professional local development environment.
- Confidently operate the command-line interface for common engineering tasks.

Semantic HTML5, Web Accessibility & Modern CSS3 Layouts

WEB102

Weeks 4–8 (60 Hours)

Crafting structured, accessible, and responsive layouts using modern CSS...

CURRICULUM TOPICS & PRACTICAL LABS

Semantic HTML5, SEO Foundations & WCAG Accessibility

Tools: Lighthouse W3C Validator axe DevTools

Constructing meaningful document outlines, SEO metadata, Open Graph tags, ARIA attributes, and accessible forms.

- KEY CONCEPTS:
- Semantic tags (<header>, <main>, <article>, <aside>)
 - Form inputs & native validation
 - ARIA roles & keyboard navigation
 - Core Web Vitals

 **Hands-on Lab Exercise:** Refactoring an unsemantic website to achieve 100/100 on Lighthouse SEO and Accessibility audits.

Modern CSS3: Box Model, Flexbox & 2D CSS Grid

Tools: CSS Grid Generator Flexbox Froggy Polypane

Deep dive into CSS specificity, cascading rules, multi-dimensional grid systems, flexible box layouts, and CSS custom properties (variables).

- KEY CONCEPTS:
- CSS Box Model & box-sizing
 - Flexbox axis alignment & wrapping
 - CSS Grid template areas & fr units
 - CSS Variables for dynamic theming

 **Hands-on Lab Exercise:** Building a responsive 12-column editorial magazine layout without external frameworks.

Responsive Web Design, Media Queries & Micro-Animations

Tools: Figma Chrome Device Emulation CSS Triggers

Mobile-first philosophy, fluid typography with clamp(), breakpoint architecture, keyframe animations, and CSS transforms.

- KEY CONCEPTS:
- Mobile-first media queries
 - Fluid units (vw, vh, rem, ch, clamp)

Transitions & @keyframes

GPU-accelerated transforms



Hands-on Lab Exercise: Creating a fully adaptive, animated agency landing page translating directly from a Figma design spec.



STUDENT COMPETENCIES & MEASURABLE OUTCOMES

- Structure complex documents with pristine semantic hierarchy and accessibility compliance.
- Implement engaging CSS micro-interactions with smooth 60fps animations.
- Build pixel-perfect responsive layouts adapted to mobile, tablet, desktop, and ultra-wide screens.

Developer Superpowers: Chrome DevTools, Inspect & Chrome Extensions

DEV103

Weeks 9–11 (36 Hours)

Mastering browser inspection, CSS live editing, DOM breakpoints, network...

CURRICULUM TOPICS & PRACTICAL LABS

Mastering the Elements & Inspect Tab

Tools:

Chrome DevTools

Firefox Developer Edition

Live DOM tree modification, box-model debugging, computed styles inspection, pseudo-state forced triggers (:hover, :active), and grid/flex overlays.

KEY CONCEPTS:

Live DOM inspection

Computed properties cascade

Forcing element states

Flex/Grid visual debugging badges



Hands-on Lab Exercise: Reverse engineering the layout and design tokens of a live top-tier SaaS website using Inspect.

Console, Network Tab, Sources & Client Storage Debugging

Tools:

DevTools Network Throttling

DevTools Memory Profiler

Console API methods (console.table, console.time), network waterfall analysis, payload inspection, conditional breakpoints, and localStorage/cookie inspection.

KEY CONCEPTS:

Network waterfall & throttling (3G simulation)

XHR/Fetch filter

Sources tab & step-over breakpoints

Application tab: localStorage, sessionStorage, IndexedDB



Hands-on Lab Exercise: Diagnosing and fixing 5 intentional performance bugs, broken API payloads, and memory leaks in a demo app.

Building & Publishing Custom Chrome Extensions

Tools:

Chrome Extension Manager

WebExtension APIs

Understanding Manifest V3, background service workers, content scripts, popup UI, permissions, and Chrome Storage API.

KEY CONCEPTS:

Manifest.json V3 structure

Content Scripts DOM injection

Popup action UI

Chrome runtime messaging



Hands-on Lab Exercise: Developing a 'Developer Quick-Inspector & Color Palette Saver' Chrome extension from scratch and running it in Developer Mode.

**STUDENT COMPETENCIES & MEASURABLE OUTCOMES**

- Rapidly pinpoint layout glitches and network bottlenecks using browser DevTools.
- Architect, package, and test production Chrome extensions using Manifest V3.
- Debug complex JavaScript runtime errors using step-through breakpoints.

JavaScript Core: Algorithms, DOM Manipulation & Event Systems

JS104

Weeks 12–18 (72 Hours)

Learn imperative and functional programming, data structures, control flow,...

CURRICULUM TOPICS & PRACTICAL LABS

JavaScript Fundamentals & Core Language Mechanics

Tools: Node.js REPL Quokka.js JSFiddle

Variables (let, const, var), primitive vs reference types, type coercion, operators, control flow (if-else, switch, loops), and pure functions.

KEY CONCEPTS: Data types & memory stack vs heap Arrow functions & default params

Scope chain & lexical scope

Closures & IIFE

 **Hands-on Lab Exercise:** Writing a suite of 20 algorithmic utility functions (sorting, deduplication, deep clone, string transforms).

Arrays, Objects & Functional Array Methods

Tools: VS Code Debugger TypeScript Playgrounds

Object manipulation, destructuring, spread/rest operators, and functional pipelines using map(), filter(), reduce(), find(), and some().

KEY CONCEPTS: Immutability principles Object.keys/values/entries

Array transformation pipelines

Chaining array methods

 **Hands-on Lab Exercise:** Building an in-memory data processing engine for a financial transaction dataset.

DOM Traversal, Manipulation & Custom Event Handling

Tools: Browser DOM APIs Event Listeners Inspector

Selecting elements (querySelector, closest), creating dynamic nodes, event delegation, bubbling/capturing, keyboard navigation, and form lifecycle.

KEY CONCEPTS: DOM tree traversal Event bubbling & event delegation

preventDefault & stopPropagation

FormData API & validation

🔥 **Hands-on Lab Exercise:** Creating an interactive Kanban board with drag-and-drop, inline editing, and validation without external libraries.

Browser Storage & Asynchronous Foundations

Tools: DevTools Storage Tab JSONLint

Persisting data with localStorage & sessionStorage, JSON parsing/stringifying, setInterval/setTimeout, and introduction to Promises.

KEY CONCEPTS: JSON serialization Storage event synchronization

Microtask vs Macrotask queue

Basic Promise creation & handling

🔥 **Hands-on Lab Exercise:** Building an offline-capable Personal Habit & Task Tracker with persistent local analytics.

✅ STUDENT COMPETENCIES & MEASURABLE OUTCOMES

- Write clean, modular, and performant vanilla JavaScript adhering to modern standards.
- Dynamically manipulate the DOM and handle complex user interactions gracefully.
- Build resilient client-side state managers utilizing browser storage APIs.

Version Control Systems: Professional Git, GitHub & GitLab Workflows

GIT105

Weeks 19–22 (48 Hours)

Industry-standard source code management, branching strategies, merge...

CURRICULUM TOPICS & PRACTICAL LABS

Git Internals, Staging, Commits & History Inspection

Tools: Git CLI GitKraken VS Code Source Control

Understanding Git's DAG architecture, staging area, commit anatomy, commit conventions (Conventional Commits), git log, diff, and stash.

KEY CONCEPTS: Git object store (blobs, trees, commits)

Conventional Commit messages git stash & git restore Inspecting diffs

 **Hands-on Lab Exercise:** Creating a local repository, tracking versions, and rolling back bad commits safely.

Branching, Merging, Rebasing & Conflict Resolution

Tools: Git Visualizer GitHub Desktop

Feature-branch workflows, Git Flow, merge strategies (fast-forward vs 3-way merge), interactive rebase, and resolving complex conflicts.

KEY CONCEPTS: git branch & git switch git merge vs git rebase

Merge conflict markers & 3-way diffs git cherry-pick & git reset

 **Hands-on Lab Exercise:** Simulating a 3-developer team repository with intentional concurrent edits and resolving conflicts step-by-step.

Remote Repositories: GitHub / GitLab & Collaboration Lifecycle

Tools: GitHub CLI (gh) GitHub Projects GitLab CI

Configuring SSH keys, remote upstreams, Pull Requests (PRs), code reviews, issues, milestones, project boards, and GitHub Pages deployment.

KEY CONCEPTS: SSH authentication Forking & upstream remotes

Pull Request templates & review comments GitHub Pages static hosting

 **Hands-on Lab Exercise:** Contributing to an open-source style team repository with structured PR reviews, approvals, and automated GitHub Pages release.

✓ **STUDENT COMPETENCIES & MEASURABLE OUTCOMES**

- Execute full version control lifecycles using the Git command line.
- Collaborate on multi-person engineering teams using GitHub Pull Requests and code reviews.
- Comfortably resolve merge conflicts without losing work.

Semester 1 Capstone Project & Evaluation Defense

SEM1 - CAP

Weeks 23–24 (48 Hours)

Consolidate all Semester 1 skills into two production-grade deliverables...

CURRICULUM TOPICS & PRACTICAL LABS

Capstone Project Development & Optimization

Tools: Lighthouse BrowserStack GitHub Pages

End-to-end development of a responsive multi-page web application accompanied by a companion Chrome extension tool.

KEY CONCEPTS: Code structure modularity

Lighthouse score optimization (>90 on all metrics)

Accessibility audit

Cross-browser testing

 **Hands-on Lab Exercise:** Complete build, git tag releases, and write comprehensive README documentation.

✓ **STUDENT COMPETENCIES & MEASURABLE OUTCOMES**

- Deploy two production-grade client-side applications live on the web.
- Present architecture and defend code quality in a technical review.



Semester 1 Capstone Projects & Portfolio Proof of Work

MILESTONE PROJECT

Project 01

Capstone 1A: Responsive SaaS Product Portal & Interactive Showcase

A multi-page responsive corporate web application with dynamic pricing calculator, interactive feature tabs, contact validation engine, and dark/light theme switcher.

SYSTEM ARCHITECTURE:

Semantic HTML5

CSS Grid & Flexbox

Modular Vanilla JS

LocalStorage Theme State

REQUIRED DELIVERABLES:

- ✓ Live GitHub Pages URL
- ✓ Semantic Markup & a11y 100 Score
- ✓ Cross-device responsive test report

Tools Mastered:

VS Code • Chrome DevTools • Git & GitHub

MILESTONE PROJECT

Project 02

Capstone 1B: 'DevLens' Chrome Productivity & Web Inspector Extension

A Manifest V3 Chrome Extension that injects tools into any webpage to inspect font stacks, color palettes, heading hierarchies, image sizes, and asset loading speed.

SYSTEM ARCHITECTURE:

Manifest V3

Content Scripts

Chrome Storage API

Popup UI & Badge Counter

REQUIRED DELIVERABLES:

- ✓ Packaged .crx Extension
- ✓ Source Code in GitHub Repo
- ✓ 3-Minute Video Demo of Live Extension

Tools Mastered:

Chrome DevTools • Manifest V3 • VS Code

SEMESTER 1 WEEKLY ROADMAP & MILESTONE SCHEDULE

Timeline	Phase Name	Core Learning Focus	Deliverable Milestone
Weeks 1–3	Phase 1: Computing Core	Architecture, DNS, HTTP, Terminal & VS Code setup	✔ Scaffolded
Weeks 4–8	Phase 2: HTML5 & CSS3 Mastery	Semantic markup, Flexbox, Grid, fluid responsive design	✔ Pixel-Perfect
Weeks 9–11	Phase 3: DevTools & Chrome Extensions	Inspect, Network tab, debugging, building Chrome extensions	✔ Published
Weeks 12–18	Phase 4: JavaScript Programming	Core JS, DOM manipulation, event systems, LocalStorage	✔ Interactive
Weeks 19–22	Phase 5: Git & Team Workflows	Branching, merging, PR reviews, GitHub & GitLab	✔ Collaborative
Weeks 23–24	Phase 6: Sem 1 Capstone Defense	Final project builds, performance tuning, live defense	✔ Deployed & Defended

SEMESTER 2 • INTERMEDIATE ENGINEERING

Months 7–12 • Weeks 25–48 • ~300 Practical Hours

Modern Frontend, Backend Engineering & Database Systems

From Client Interfaces to Dynamic Full-Stack Client-Server Architectures

Transitions students from static web creation into professional software development: building reactive single-page applications with React.js, crafting robust RESTful backends with Node.js and Express, and engineering relational (PostgreSQL) and document (MongoDB) databases.

 **SEMESTER CORE GOAL:** Students architect and deploy a complete client-server application featuring an interactive React frontend, custom Express API endpoints, SQL/NoSQL databases with relational schemas, and secure authentication.

5 Modules
2 Capstone Projects

FRONTEND FRAMEWORK

- React.js
- JSX
- React Hooks
- Context API
- React Router v6
- Tailwind CSS

BACKEND RUNTIME

- Node.js
- Express.js
- REST APIs
- Middleware
- Zod Validation
- JWT Auth

DATABASES

- PostgreSQL
- SQL Queries
- Prisma ORM
- MongoDB
- Mongoose ODM

TESTING & TOOLING

- Postman
- Thunder Client
- npm/pnpm
- ESLint
- Environment Config

 Detailed Course Modules & Units

Showing 5 of 5 Modules

Advanced JavaScript & Asynchronous Architecture

JS201

Weeks 25–28 (48 Hours)

Master the JavaScript Event Loop, Callbacks, Promises, Async/Await, Fetch API,...

CURRICULUM TOPICS & PRACTICAL LABS

The JavaScript Runtime, Event Loop & Concurrency

Tools: Loupe Visualizer Node.js Performance Hooks

How V8 executes code: Call stack, Web APIs, Task Queue, Microtask Queue (Promises), and non-blocking I/O execution model.

KEY CONCEPTS: V8 engine internals Call Stack vs Task Queue
Microtask vs Macrotask prioritization Memory leaks & garbage collection

 **Hands-on Lab Exercise:** Predicting and profiling event loop execution sequences using Loupe and browser performance profiler.

Asynchronous JavaScript: Promises, Async/Await & Error Handling

Tools: Postman Mock Server Public REST APIs (PokeAPI, GitHub API)

Promise creation, Promise.all, Promise.allSettled, Promise.race, async/await syntax, try/catch/finally blocks, and custom error classes.

KEY CONCEPTS: Promise lifecycle (pending, fulfilled, rejected)
Sequential vs parallel async execution Custom Error inheritance
Graceful error recovery

 **Hands-on Lab Exercise:** Building a resilient multi-source data fetcher with auto-retry mechanisms and timeout fallback.

Modern ES6+ Modules, Fetch API & HTTP Request Handling

Tools: cURL Fetch Sandbox JSONPlaceholder

ES Module import/export syntax, Fetch API (GET, POST, PUT, DELETE), setting headers, Bearer tokens, CORS handling, and streaming responses.

KEY CONCEPTS:

Named vs default exports

Request & Response objects

Handling CORS errors

Blob and JSON payload parsing



Hands-on Lab Exercise: Constructing a reusable HTTP Client class with request interceptors and centralized error logging.

**STUDENT COMPETENCIES & MEASURABLE OUTCOMES**

- Diagnose and solve complex asynchronous timing bugs.
- Architect reusable API communication modules.
- Write clean, async/await code with bulletproof error handling.

Modern React.js: Component Architecture, Hooks & State Management

REACT202

Weeks 29–35 (84 Hours)

Declarative UI engineering with React: JSX, component lifecycle, useState,...

CURRICULUM TOPICS & PRACTICAL LABS

React Fundamentals: JSX, Virtual DOM & Component Hierarchy

Tools: React DevTools Vite ESLint React Plugin

How React renders UI, Virtual DOM reconciliation, Fiber architecture basics, props vs state, unidirectional data flow, and conditional rendering.

KEY CONCEPTS: Virtual DOM diffing algorithm Pure components & immutability

Lifting state up Props drilling resolution

 **Hands-on Lab Exercise:** Building an interactive real-time dashboard layout with modular components.

Mastering React Hooks & Side Effects

Tools: React Profiler Web Audio API

In-depth useState, useEffect dependency arrays, cleanup functions, useRef for DOM access and mutable values, useMemo and useCallback for optimization.

KEY CONCEPTS: Effect cleanup lifecycles Avoiding infinite re-renders

useRef for persistence without re-render Memoization tradeoffs

 **Hands-on Lab Exercise:** Engineering a full audio synthesizer / music pad UI with real-time waveform updates and keyboard shortcuts.

Global State Management with Context API & Custom Hooks

Tools: React DevTools Component Tree

Creating custom reusable hooks (useFetch, useLocalStorage, useDebounce), Context API providers, reducers with useReducer.

KEY CONCEPTS: Custom hook abstraction Context Provider pattern

useReducer for complex state machines State selector optimization



Hands-on Lab Exercise: Building a complete Shopping Cart & Checkout state system with discount coupons and local persistence.

Client-Side Routing with React Router v6 & Tailwind CSS UI

Tools: React Router v6 Tailwind CSS IntelliSense

Nested routes, dynamic URL parameters, loaders/actions, protected routes (auth guards), 404 handlers, and rapid UI development with Tailwind CSS.

KEY CONCEPTS:

BrowserRouter & Routes

useNavigate & useParams

Protected Route wrapper

Utility-first CSS styling



Hands-on Lab Exercise: Building a multi-view streaming media catalog with search, filter, detail pages, and favorite lists.



STUDENT COMPETENCIES & MEASURABLE OUTCOMES

- Architect large-scale, performant Single Page Applications (SPAs) with React.
- Implement declarative routing, authentication guards, and responsive Tailwind UI.
- Extract clean, reusable custom hooks for data fetching and UI state.

Backend Engineering: Node.js, Express.js & RESTful API Design

NODE203

Weeks 36–41 (72 Hours)

Server-side programming with Node.js, Express routing, middleware...

CURRICULUM TOPICS & PRACTICAL LABS

Node.js Core Architecture & Built-in Modules

Tools: Node.js v20+ dotenv tsx/nodemon

Node.js runtime, process object, File System (fs/promises), path, events, buffer, streams, and creating an HTTP server from scratch.

KEY CONCEPTS: CommonJS vs ES Modules fs module file manipulation

Readable/Writable streams

Process environment variables



Hands-on Lab Exercise: Building a CLI log analyzer tool that reads massive log files via streams and outputs JSON summaries.

Express.js Framework & Layered Architecture

Tools: Express.js Postman Thunder Client

Express application structure, routers, controllers, service layers, request parameters (params, query, body), and JSON response formatting.

KEY CONCEPTS: MVC / Controller-Service-Repository pattern

Express Router modularization

RESTful resource naming conventions

HTTP status code semantics



Hands-on Lab Exercise: Architecting a modular REST API for a University Course Management system.

Express Middleware, Request Validation & Centralized Error Handling

Tools: Zod Morgan Helmet

Custom middleware pipeline, authentication checks, request logging (Morgan), schema validation with Zod, and global error handling middleware.

KEY CONCEPTS: Middleware next() chain Schema validation with Zod

Global error handler middleware

Custom AppError exception hierarchy



Hands-on Lab Exercise: Implementing a complete input sanitization and validation pipeline blocking malformed payloads.



STUDENT COMPETENCIES & MEASURABLE OUTCOMES

- Develop structured, maintainable backend APIs using Express and Node.js.
- Design industry-standard RESTful APIs adhering to HTTP best practices.
- Implement robust middleware pipelines for validation, logging, and error handling.

Database Engineering: Relational SQL (PostgreSQL) & NoSQL (MongoDB)

DB204

Weeks 42–46 (60 Hours)

Database modeling, normalization, SQL queries, relational joins, indexing, ACID...

CURRICULUM TOPICS & PRACTICAL LABS

Relational Database Design & SQL Mastery (PostgreSQL)

Tools: PostgreSQL pgAdmin DBeaver TablePlus

Relational modeling, primary & foreign keys, constraints, normalization (1NF, 2NF, 3NF), SELECT queries, GROUP BY, aggregations, JOINS (INNER, LEFT, FULL).

- KEY CONCEPTS:
- Entity-Relationship (ER) Diagrams
 - Normalization rules
 - Complex JOIN queries & subqueries
 - ACID properties & transactions

 **Hands-on Lab Exercise:** Designing an E-commerce relational schema in PostgreSQL, executing complex reporting queries and transactions.

Database Access with Modern ORMs: Prisma ORM

Tools: Prisma Studio Prisma CLI

Prisma schema definition, database migrations, model relations (1-1, 1-many, many-many), type-safe CRUD queries, and seeds.

- KEY CONCEPTS:
- Prisma schema syntax
 - Database migrations workflow
 - Type-safe query builder
 - Connection pooling

 **Hands-on Lab Exercise:** Connecting an Express backend to PostgreSQL via Prisma ORM with automated migrations.

Document Databases: MongoDB & Mongoose ODM

Tools: MongoDB Atlas MongoDB Compass Mongoose

NoSQL document model, BSON format, MongoDB Atlas cloud setup, Mongoose schemas, validation, middleware hooks, and aggregation pipelines.

- KEY CONCEPTS:
- Document vs Relational paradigm
 - Mongoose schema & virtuals
 - Embedding vs Referencing documents
 - Aggregation pipeline (\$match, \$group, \$sort)



Hands-on Lab Exercise: Building a social media content backend with nested comments, likes, and tag aggregation using MongoDB.



STUDENT COMPETENCIES & MEASURABLE OUTCOMES

- Model relational and document database schemas suited to business requirements.
- Select the right database technology (SQL vs NoSQL) for specific system constraints.
- Write optimized SQL queries and leverage modern ORMs for type-safe database access.

Semester 2 Capstone: Integrated Full-Stack Application

SEM2-CAP

Weeks 47–48 (36 Hours)

Build, integrate, test, and defend a complete end-to-end full-stack...

CURRICULUM TOPICS & PRACTICAL LABS

Full-Stack Integration, API Wiring & Live Deployment

Tools: Vercel Render Railway Postman

Connecting React state to backend REST endpoints, handling loading/error states, CORS configuration, seed data, and deployment.

KEY CONCEPTS: Full-stack repository setup Environment configuration (.env)

Cross-Origin Resource Sharing Database seeding



Hands-on Lab Exercise: Deploying backend API to Render/Railway and frontend to Vercel.



STUDENT COMPETENCIES & MEASURABLE OUTCOMES

- Deliver a production-ready full-stack software application.
- Defend API architecture and database schema in technical oral review.



Semester 2 Capstone Projects & Portfolio Proof of Work

MILESTONE PROJECT

Project 01

Capstone 2A: Full-Stack E-Commerce & Inventory Management System

Complete full-stack store with customer catalog, shopping cart, customer checkout, orders history, and admin panel for product inventory & sales analytics.

SYSTEM ARCHITECTURE:

- React.js Frontend
- Node.js & Express API
- PostgreSQL & Prisma ORM
- Tailwind CSS

REQUIRED DELIVERABLES:

- ✔ Live Full-Stack App
- ✔ Documented Postman Collection
- ✔ Database ER Diagram & Schema Migration Scripts

Tools Mastered: React • Express • PostgreSQL • Prisma • Postman

MILESTONE PROJECT

Project 02

Capstone 2B: Multi-User Project Collaboration & Task Workspace

Trello/Asana style project workspace with board columns, task assignments, activity logs, file attachments, and user role management.

SYSTEM ARCHITECTURE:

- React.js
- Express REST API
- MongoDB & Mongoose
- Context API State

REQUIRED DELIVERABLES:

- ✔ Live Deployed Web App
- ✔ Full REST API Swagger/OpenAPI Spec
- ✔ GitHub Repo with CI testing

Tools Mastered: React • Node.js • MongoDB • Tailwind CSS

SEMESTER 2 WEEKLY ROADMAP & MILESTONE SCHEDULE

Timeline	Phase Name	Core Learning Focus	Deliverable Milestones
Weeks 25–28	Phase 1: Async JS & Event Loop	Promises, Async/Await, V8 Event	✔ Resilient Async

Timeline	Phase Name	Core Learning Focus	Deliverable Milestones
		loop, Fetch API	
Weeks 29–35	Phase 2: React.js & SPA Development	JSX, Hooks, Context, React Router v6, Tailwind	✔ Feature-Complete Frontend
Weeks 36–41	Phase 3: Node.js & Express REST APIs	Server architecture, Routing, Middleware, Zod, Errors	✔ Production-Ready Backend
Weeks 42–46	Phase 4: Database Engineering	PostgreSQL, SQL queries, Prisma ORM, MongoDB	✔ Normalized Relational Schema
Weeks 47–48	Phase 5: Full-Stack Capstone Defense	End-to-end integration, deployment, live defense	✔ Deployed Full-Stack Application

SEMESTER 3 • ADVANCED FULL-STACK & CLOUD

Months 13–18 • Weeks 49–72 • ~300 Practical Hours

Advanced Full-Stack, Real-Time Systems, Cloud & DevOps

Enterprise Frameworks, Authentication Security, WebSockets, Docker & Cloud Pipelines

Elevates developer skills to enterprise grade: mastering Next.js (SSR, Server Actions), robust authentication systems (JWT, OAuth2, RBAC), real-time WebSockets, Redis caching, third-party payment gateways (Stripe), Docker containerization, and automated CI/CD deployment pipelines.



SEMESTER CORE GOAL: Students engineer resilient, secure, and scalable cloud-hosted full-stack applications with real-time bidirectional communication, containerized microservices, and continuous deployment.

5 Modules

2 Capstone Projects

FULL-STACK FRAMEWORK

Next.js (App Router)

React Server Components

Server Actions

TypeScript

Tailwind CSS

AUTH & SECURITY

JWT

OAuth 2.0

Role-Based Access (RBAC)

Bcrypt

CORS & Helmet

Rate Limiting

REAL-TIME & CACHING

WebSockets

Socket.io

Server-Sent Events

Redis Caching

Pub/Sub

CLOUD & DEVOPS

Docker

Docker Compose

GitHub Actions CI/CD

Nginx Reverse Proxy

AWS/VPS Cloud

Detailed Course Modules & Units

Showing 5 of 5 Modules

Enterprise Full-Stack with Next.js & TypeScript

NEXT301

Weeks 49–54 (72 Hours)

Next.js App Router, React Server Components (RSC), Client Components,...

CURRICULUM TOPICS & PRACTICAL LABS

TypeScript for Full-Stack Developers

Tools: TypeScript Compiler ts-node TS Playground

Static type checking, interfaces, type aliases, generics, union/intersection types, type narrowing, and typing Express/React codebases.

KEY CONCEPTS: Generics in functions & interfaces Utility types (Partial, Pick, Omit)

Strict type checking Zod-to-TypeScript type inference

 **Hands-on Lab Exercise:** Migrating an entire JavaScript React/Express codebase to strict TypeScript with zero 'any' types.

Next.js App Router & React Server Components (RSC)

Tools: Next.js 14+ Vercel Analytics

File-system routing (page.tsx, layout.tsx, loading.tsx, error.tsx), RSC benefits, Client component boundaries ('use client'), and metadata management.

KEY CONCEPTS: Server vs Client components paradigm Streaming UI with Suspense

Nested layouts & route groups Dynamic SEO metadata generation

 **Hands-on Lab Exercise:** Building a lightning-fast News & Media publishing platform utilizing server components and streaming.

Server Actions, Mutations & Data Fetching Caching

Tools: Next.js DevTools Turbopack

Direct server mutations with Server Actions, revalidatePath, revalidateTag, cache controls, dynamic vs static rendering, and optimistic UI updates.

KEY CONCEPTS: Server Actions form handling Incremental Static Regeneration (ISR)

Optimistic UI with useOptimistic Cache tags invalidation



Hands-on Lab Exercise: Developing an interactive Product Review and Rating portal with instant optimistic updates and server validation.



STUDENT COMPETENCIES & MEASURABLE OUTCOMES

- Build high-performance, SEO-optimized production applications with Next.js.
- Implement optimal data fetching and caching architectures with Server Actions.
- Write clean, type-safe full-stack code using TypeScript.

Advanced Authentication, Authorization & Web Security

SEC302

Weeks 55–58 (48 Hours)

Industry-standard authentication: Password hashing, JSON Web Tokens (JWT),...

CURRICULUM TOPICS & PRACTICAL LABS

Cryptographic Password Hashing & JWT Architecture

Tools: `jwt.io` `Postman` `Crypto module`

Salting & hashing with Bcrypt/Argon2, Access Token vs Refresh Token lifecycle, HTTP-only secure cookies, and token revocation strategies.

KEY CONCEPTS: `Bcrypt work factors & salting` `JWT payload signature & expiration`
`Silent token refresh with HTTP-only cookies` `XSS & CSRF token storage tradeoffs`

 **Hands-on Lab Exercise:** Building an enterprise Auth microservice with automatic refresh token rotation and multi-device session revocation.

OAuth 2.0 / OpenID Connect & Social Logins

Tools: `Google Cloud Console` `GitHub Developer Settings` `Auth.js`

OAuth2 authorization code grant flow, Google/GitHub social login integration, NextAuth.js / Auth.js, and user identity profile merging.

KEY CONCEPTS: `Authorization Code flow` `Client ID & Client Secret security`
`Scopes and consent screens` `Linking multi-provider identities`

 **Hands-on Lab Exercise:** Integrating Google & GitHub single-click OAuth authentication into a Next.js application.

Role-Based Access Control (RBAC) & OWASP Security Defenses

Tools: `OWASP ZAP` `Helmet` `express-rate-limit`

Granular permissions, user roles (Admin, Member, Guest), route middleware guards, SQL injection prevention, XSS mitigation, CSRF protection, rate limiting, and CORS.

KEY CONCEPTS: `RBAC permission matrices` `OWASP Top 10 vulnerabilities`
`Express Rate Limit & Helmet headers` `Content Security Policy (CSP)`



Hands-on Lab Exercise: Conducting a security penetration test on an intentionally vulnerable app and patching all vulnerabilities.



STUDENT COMPETENCIES & MEASURABLE OUTCOMES

- Implement enterprise-grade JWT and OAuth2 authentication flows.
- Harden web applications against OWASP Top 10 vulnerabilities.
- Enforce role-based access control across frontend and backend services.

Real-Time Systems, WebSockets & High-Performance Caching

RT303

Weeks 59–63 (60 Hours)

Bidirectional real-time communication with WebSockets (Socket.io), Server-Sen...

CURRICULUM TOPICS & PRACTICAL LABS

Bidirectional WebSockets & Socket.io Architecture

Tools: Socket.io Postman WebSocket client

WebSocket handshake, full-duplex communication, Socket.io rooms, namespaces, broadcasting, connection heartbeats, and reconnection management.

KEY CONCEPTS: HTTP Upgrade handshake Socket.io rooms & private channels

Event acknowledgment callbacks

Managing connection dropouts



Hands-on Lab Exercise: Building a multi-room real-time collaborative chat application with typing indicators, online status, and read receipts.

In-Memory Caching & Pub/Sub with Redis

Tools: Redis CLI RedisInsight ioredis

Redis data structures (strings, hashes, lists, sets), cache-aside pattern, TTL expiration, cache invalidation strategies, and Redis Pub/Sub for horizontal scaling.

KEY CONCEPTS: Cache-aside vs write-through Time-To-Live (TTL) strategies

Cache stampede mitigation

Redis Pub/Sub message broker



Hands-on Lab Exercise: Adding a high-throughput Redis caching layer to an API, reducing database latency from 250ms to 4ms.

Media Storage & Third-Party APIs (Stripe Payments, S3, Email)

Tools: Stripe CLI AWS S3 Resend / SendGrid

Direct cloud file uploads (AWS S3 / Cloudinary) with signed URLs, Stripe payment checkout & webhooks, and transactional emails with Resend.

KEY CONCEPTS: S3 presigned upload URLs

Stripe payment intents & webhook validation

Idempotency keys

Transactional email templates



Hands-on Lab Exercise: Building a complete digital product checkout flow with Stripe webhooks and automated PDF receipt email dispatch.



STUDENT COMPETENCIES & MEASURABLE OUTCOMES

- Architect low-latency real-time applications using WebSockets.
- Implement high-performance caching layers with Redis.
- Integrate production third-party services like Stripe payments and S3 media storage.

Containerization, DevOps & Automated CI/CD Pipelines

OPS304

Weeks 64–70 (72 Hours)

Docker containers, multi-stage Dockerfiles, Docker Compose multi-container...

CURRICULUM TOPICS & PRACTICAL LABS

Docker

Tools: Docker Desktop Docker CLI Dive image analyzer

Containerization & Multi-Stage Builds

Containers vs Virtual Machines, Dockerfile instructions, layer caching, multi-stage builds for minimal production images, and .dockerignore.

KEY CONCEPTS: Docker images & container lifecycles Multi-stage build optimization

Container networking & port mapping Volume persistence

 **Hands-on Lab Exercise:** Containerizing a Next.js frontend, Node.js API, and PostgreSQL database into lightweight production Docker images.

Multi-Service Orchestration with Docker Compose

Tools: Docker Compose Docker Volumes

Defining multi-container architectures in docker-compose.yml: App, DB, Redis, Nginx, environment variable propagation, and healthchecks.

KEY CONCEPTS: docker-compose service dependencies (depends_on)

Named volumes & custom networks Healthcheck probes

One-command local dev spin-up

 **Hands-on Lab Exercise:** Setting up a 1-command reproducible full-stack development environment with Docker Compose.

Linux Server Administration, Nginx Reverse Proxy & SSL

Tools: Ubuntu Linux Nginx Certbot OpenSSH

Configuring Ubuntu cloud VPS (AWS EC2 / DigitalOcean), SSH keys, UFW firewalls, systemd processes, Nginx reverse proxy configuration, and Let's Encrypt SSL.

KEY CONCEPTS: SSH hardened access Nginx server blocks & proxy_pass

Certbot automated SSL certificates Zero-downtime reload with PM2/Docker



Hands-on Lab Exercise: Provisioning a live Linux VPS, configuring Nginx to reverse proxy port 3000 to custom domains with HTTPS.

CI/CD Automation with GitHub Actions

Tools: GitHub Actions SSH Action Docker Hub

Automated Continuous Integration and Continuous Deployment: Workflow YAML, automated lint/test triggers on Pull Requests, and automated Docker build and deploy.

KEY CONCEPTS: GitHub Actions triggers (push, pull_request)

Runner environments & matrix testing

GitHub Secrets management

Automated deployment webhooks



Hands-on Lab Exercise: Creating a production CI/CD pipeline that runs automated tests and deploys code to production automatically on git push to main.

✓ STUDENT COMPETENCIES & MEASURABLE OUTCOMES

- Package full-stack applications into optimized production Docker containers.
- Provision and secure Linux cloud servers with Nginx reverse proxy and SSL.
- Build automated CI/CD pipelines deploying tested code without manual intervention.

Semester 3 Capstone: Production Cloud SaaS Platform

SEM3-CAP

Weeks 71–72 (48 Hours)

Design, containerize, and deploy a production-grade multi-tenant SaaS...

CURRICULUM TOPICS & PRACTICAL LABS

Production SaaS Deployment & Live Systems Defense

Tools: k6 Load Tester Sentry Vercel / Cloud VPS

Finalizing cloud deployment, load testing, uptime monitoring, error logging with Sentry, and presenting the architecture.

KEY CONCEPTS: Load testing with k6 Error tracking with Sentry
Zero-downtime updates Technical architecture presentation

 **Hands-on Lab Exercise:** Deploying the complete SaaS system to live domain with active SSL, running load tests and presenting to an engineering panel.

✓ STUDENT COMPETENCIES & MEASURABLE OUTCOMES

- Deploy an enterprise-grade cloud SaaS application with real users.
- Defend cloud infrastructure, security, and scalability decisions.

Semester 3 Capstone Projects & Portfolio Proof of Work

MILESTONE PROJECT

Project 01

Capstone 3A: Real-Time Multi-Room Collaborative Whiteboard & Chat

Figma-like live whiteboard canvas with multi-user cursor tracking, shape drawing, real-time audio/chat channels, and state synchronization.

SYSTEM ARCHITECTURE:

Next.js (App Router)

Socket.io WebSockets

Redis Pub/Sub & Caching

PostgreSQL

Docker Compose

REQUIRED DELIVERABLES:

- ✔ Live Hosted App with Custom Domain
- ✔ Stress Test Report with 100+ simulated concurrent connections
- ✔ CI/CD Workflow Config

Tools Mastered: Next.js • Socket.io • Redis • Docker • GitHub Actions

MILESTONE PROJECT

Project 02

Capstone 3B: Subscription SaaS Platform with Stripe Billing & RBAC

Production SaaS platform featuring Tiered subscriptions (Free, Pro, Enterprise), Stripe webhook billing, team workspaces, RBAC permissions, and S3 file asset management.

SYSTEM ARCHITECTURE:

TypeScript Full-Stack

Next.js 14

Stripe API & Webhooks

Prisma + PostgreSQL

AWS S3

Dockerized VPS

REQUIRED DELIVERABLES:

- ✔ Live Production URL
- ✔ Automated GitHub Actions Deployment Pipeline
- ✔ Security Audit Checklist & Sentry Error Dashboard

Tools Mastered: Next.js • TypeScript • Stripe • AWS S3 • Docker • Nginx

SEMESTER 3 WEEKLY ROADMAP & MILESTONE SCHEDULE

Timeline	Phase Name	Core Learning Focus	Deliverable Milestones
Weeks 49–54	Phase 1: Next.js & TypeScript	App Router, RSC, Server Actions, strict TypeScript	✔ Full-Stack Next.js
Weeks 55–58	Phase 2: Auth, Security & RBAC	JWT, OAuth2, Refresh tokens, RBAC, OWASP Top 10	✔ Hardened Multi-T

Timeline	Phase Name	Core Learning Focus	Deliverable Milestone
Weeks 59–63	Phase 3: Real-Time, Redis & Stripe	WebSockets, Socket.io, Redis caching, Stripe billing	✔ Real-Time Billing
Weeks 64–70	Phase 4: Docker, Linux & CI/CD	Docker Compose, VPS, Nginx, SSL, GitHub Actions	✔ Automated Contin
Weeks 71–72	Phase 5: Cloud SaaS Defense	SaaS final build, load testing, live architecture defense	✔ Deployed Live Pro

SEMESTER 4 • INDUSTRY PROFESSIONAL & AI

Months 19–24 • Weeks 73–96 • ~300 Practical Hours

Professional Architecture, Mobile, AI Systems & Industry Launchpad

Enterprise Software Engineering, Flutter Mobile, Applied AI/LLMs & Job Readiness

Transforms students into versatile, senior-ready software engineers: mastering Clean Architecture and Design Patterns, comprehensive automated testing (Unit, Integration, E2E), cross-platform mobile app development with Flutter & Dart, Generative AI SDK integrations (Gemini API, RAG, Agents), Agile project leadership, and complete technical interview mastery.

 **SEMESTER CORE GOAL:** Students build and launch their Grand Capstone (a cross-platform AI-integrated enterprise software suite) and graduate fully prepared for Software Engineer, Full-Stack Developer, and Mobile Developer roles.

6 Modules

1 Capstone Projects

SOFTWARE ARCHITECTURE

- Clean Architecture
- SOLID Principles
- Design Patterns
- Clean Code
- System Design
- Microservices

TESTING & QA

- Jest / Vitest
- React Testing Library
- Supertest API Tests
- Playwright E2E
- TDD Workflow

MOBILE ENGINEERING

- Flutter
- Dart
- Provider / Riverpod
- REST API Mobile Integration
- Firebase Push Notifications

APPLIED AI & LLMS

- Google Gemini API
- LLM Function Calling
- RAG & Vector Search
- Prompt Engineering
- AI Chatbots

CAREER & LEADERSHIP

- DSA Patterns (LeetCode)
- System Design Mock Interviews
- Portfolio & Resume
- Agile/Scrum (Jira)

Software Engineering Principles, Clean Architecture & Design Patterns

ARCH401

Weeks 73–76 (48 Hours)

Write maintainable enterprise code: SOLID principles, Clean Code practices,...

CURRICULUM TOPICS & PRACTICAL LABS

SOLID Principles & Clean Code in Practice

Tools: SonarQube Code Climate TypeScript

Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion, DRY, KISS, and code refactoring strategies.

KEY CONCEPTS: SOLID code refactoring Dependency Injection (DI) containers

Code smells identification

Cyclomatic complexity reduction

 **Hands-on Lab Exercise:** Refactoring a 1,000-line monolithic codebase into a pristine, decoupled SOLID architecture with dependency injection.

Design Patterns in Modern Software Engineering

Tools: UML Diagrammers (Mermaid.js) Draw.io

Creational (Factory, Builder, Singleton), Structural (Adapter, Decorator, Facade), and Behavioral (Observer, Strategy, Repository) patterns applied to real code.

KEY CONCEPTS: Repository pattern for data abstraction

Observer pattern for event buses

Strategy pattern for interchangeable payment engines

Factory pattern for dynamic modules

 **Hands-on Lab Exercise:** Building an extensible Multi-Provider Notification Engine (Email, SMS, Push, Slack) using Strategy and Factory design patterns.

System Design Fundamentals for Scalable Software

Tools: Excalidraw Mermaid.js System Design Primer

High-level architecture design: Monolith vs Microservices, Horizontal vs Vertical scaling, Load Balancers, Database Sharding, Read Replicas, and Message Brokers (RabbitMQ/Kafka basics).

KEY CONCEPTS: CAP Theorem & High Availability

Database read-replicas & partitioning

Message queues & asynchronous workers

API Gateway patterns



Hands-on Lab Exercise: Designing the full architectural blueprint for a ride-sharing service handling 100,000 requests/sec with diagram specs.



STUDENT COMPETENCIES & MEASURABLE OUTCOMES

- Apply SOLID principles and design patterns to solve complex enterprise problems.
- Refactor legacy code into modular, maintainable software architectures.
- Design scalable, fault-tolerant distributed systems and communicate architectures effectively.

Automated Testing, Quality Assurance & Test-Driven Development (TDD)

TEST402

Weeks 77–80 (48 Hours)

Comprehensive software testing pyramid: Unit testing with Jest/Vitest,...

CURRICULUM TOPICS & PRACTICAL LABS

Unit Testing & Test-Driven Development (TDD)

Tools: Vitest Jest c8 Coverage

Writing testable code, AAA pattern (Arrange-Act-Assert), test runners (Vitest/Jest), mocks, spies, stubs, code coverage metrics (>80%), and TDD red-green-refactor cycles.

KEY CONCEPTS: Vitest assertion library Mocking external dependencies

Code coverage thresholds Test-Driven Development mindset

 **Hands-on Lab Exercise:** Developing a complete financial ledger calculator strictly utilizing TDD from scratch.

Frontend Component Testing & Backend API Testing

Tools: React Testing Library Mock Service Worker (MSW) Supertest

Testing React components with React Testing Library, user event simulation, accessibility assertions, and backend REST API testing with Supertest and test databases.

KEY CONCEPTS: Testing user interactions (screen.getByRole)

Mocking API calls (MSW - Mock Service Worker) Supertest HTTP endpoint assertions

Test database isolation & teardown

 **Hands-on Lab Exercise:** Building a comprehensive test suite for a full-stack authentication and checkout flow.

End-to-End (E2E) Testing with Playwright

Tools: Playwright Test Playwright Trace Viewer

Automating browser actions with Playwright, visual regression testing, cross-browser test suites (Chromium, Firefox, WebKit), CI integration, and test artifacts.

KEY CONCEPTS:

Playwright test fixtures

Page Object Model (POM) pattern

Handling async race conditions

Running headless E2E tests in GitHub Actions



Hands-on Lab Exercise: Writing automated E2E tests covering the complete critical user checkout journey with video recordings and failure traces.

**STUDENT COMPETENCIES & MEASURABLE OUTCOMES**

- Construct a bulletproof testing pyramid achieving high automated code coverage.
- Automate cross-browser E2E regression test suites in CI pipelines.
- Practice Test-Driven Development to produce resilient, bug-free software.

Cross-Platform Mobile Application Development (Flutter & Dart)

MOB403

Weeks 81–85 (60 Hours)

Building native mobile apps for Android and iOS using Flutter and Dart: Widg...

CURRICULUM TOPICS & PRACTICAL LABS

Dart Programming & Flutter Framework Architecture

Tools: Flutter SDK Android Studio / Emulator Dart DevTools

Dart language (null safety, OOP, async/await, streams), Flutter rendering engine, Stateless vs Stateful widgets, layout widgets (Column, Row, Stack, ListView).

KEY CONCEPTS: Dart sound null safety Widget tree composition & rebuilds
Material Design & Cupertino components Responsive mobile layouts

 **Hands-on Lab Exercise:** Building a multi-screen mobile travel discovery app with smooth Hero transitions and animations.

State Management & Device API Integration in Flutter

Tools: Riverpod Dio Hive DB

State management using Provider / Riverpod, local caching (SharedPreferences, Hive), accessing camera, geolocation, offline sync, and REST API consumption (Dio/Http).

KEY CONCEPTS: Provider / Riverpod reactive state Local storage with Hive
Geolocator & Camera device plugins Dio interceptors for mobile API tokens

 **Hands-on Lab Exercise:** Creating a mobile Field Inspection App with offline photo storage and automated background sync when online.

Firebase Integration, Push Notifications &

Tools: Firebase Console Android Keystore Tool Flutter Build

App Release Builds

Firebase Authentication, Cloud Firestore mobile client, Firebase Cloud Messaging (FCM) push notifications, and generating signed release APKs and app bundles.

KEY CONCEPTS:

Firebase Core & Auth setup

FCM push notification payloads & background handlers

Android keystore signing

Google Play Store deployment workflow



Hands-on Lab Exercise: Publishing a complete Android app APK with push notification handling and live backend connectivity.

✓ STUDENT COMPETENCIES & MEASURABLE OUTCOMES

- Develop cross-platform mobile apps for Android and iOS using Flutter.
- Implement reactive state management and integrate native hardware capabilities.
- Connect mobile clients securely to backend REST APIs and push notification servers.

Applied AI Engineering: Generative AI, Gemini APIs & LLM Integration

AI404

Weeks 86–89 (48 Hours)

Integrate modern AI capabilities into web and mobile software: Google Gemini...

CURRICULUM TOPICS & PRACTICAL LABS

Generative

Tools: Google AI Studio @google/genai SDK Gemini 2.0 / 1.5 Flash

AI

Foundations

& Gemini

SDK

Integration

How Large Language Models (LLMs) work, tokens, temperature, context windows, using the modern `@google/genai` TypeScript SDK, streaming AI responses, and multimodal inputs (text, image, audio).

KEY CONCEPTS: @google/genai SDK setup Streaming responses to UI in real-time
Multimodal prompting (Image + Text analysis) System instructions & guardrails



Hands-on Lab Exercise: Building an AI-Powered Code Reviewer and Invoice Analyzer that extracts structured JSON data from uploaded images.

Prompt Engineering, Structured JSON Outputs & Function Calling

Tools: Gemini Function Calling Zod Schema validation

Few-shot prompting, chain-of-thought, enforcing strict JSON schemas with responseSchema, tool calling (function calling) allowing LLMs to query databases and external APIs.

KEY CONCEPTS: Schema-constrained structured JSON output
LLM Function Calling declarations Executing server-side tool handlers
Looping multi-turn agent interactions



Hands-on Lab Exercise: Creating an autonomous AI Database Query Assistant that converts natural language into SQL, runs the query safely, and charts results.

Retrieval-Augmented Generation (RAG) & Vector Embeddings

Tools: pgvector LangChain / AI SDK PostgreSQL

Vector embeddings, cosine similarity, Vector databases (Pinecone / pgvector in PostgreSQL), chunking strategies, semantic search, and grounding AI responses in private documentation.

KEY CONCEPTS: Text embedding models pgvector extension in PostgreSQL

Semantic search vs keyword search RAG context injection pipeline

 **Hands-on Lab Exercise:** Developing a 'Chat with Your Company Documentation' enterprise RAG assistant with accurate source citation.

✓ STUDENT COMPETENCIES & MEASURABLE OUTCOMES

- Integrate Gemini AI models into production web and mobile applications.
- Engineer Retrieval-Augmented Generation (RAG) systems with vector databases.
- Build AI assistants utilizing structured JSON outputs and function calling tools.

Agile Leadership, Project Management & Career Launchpad

IND405

Weeks 90–94 (60 Hours)

Industry workflows: Agile/Scrum sprints with Jira/Linear, technical...

CURRICULUM TOPICS & PRACTICAL LABS

Agile Methodologies, Scrum Sprints & Team Collaboration

Tools: Jira Software Linear Confluence

Scrum framework (Sprint planning, daily standups, backlog grooming, retrospectives), user stories, acceptance criteria, story points, and task tracking on Jira/Linear.

KEY CONCEPTS: User stories & definition of done Sprint burn-down charts

PR code review etiquette Agile estimation techniques

 **Hands-on Lab Exercise:** Managing the final Capstone project in 2-week Agile sprints using Jira with daily standups and sprint reviews.

Data Structures & Algorithms (DSA) Patterns for Technical Interviews

Tools: LeetCode NeetCode HackerRank

Mastering the top 14 technical coding patterns: Two Pointers, Sliding Window, Fast & Slow Pointers, BFS/DFS Tree & Graph traversals, Dynamic Programming basics, and Big-O time/space analysis.

KEY CONCEPTS: Big-O Space/Time complexity Sliding window string/array problems

Hash map frequency counters Binary search & tree traversals

 **Hands-on Lab Exercise:** Solving and explaining 50 curated LeetCode Medium problems with optimal time/space complexity.

High-Impact Developer Portfolio, Resume & Mock Interview Mastery

Tools: GitHub Pages Pramp / Interviewing.io LinkedIn

Building an elite developer portfolio website, optimizing GitHub repositories (READMEs, architecture diagrams, live demos), LinkedIn optimization, behavioral

interview preparation (STAR method), and system design defense.

KEY CONCEPTS: Portfolio architecture presentation

STAR method for behavioral questions

Whiteboard coding communication

Freelancing & salary negotiation strategies



Hands-on Lab Exercise: Conducting 2 full-length simulated technical interviews (Coding + System Design + Behavioral) with industry mentors.



STUDENT COMPETENCIES & MEASURABLE OUTCOMES

- Navigate professional Agile/Scrum engineering team environments confidently.
- Present a stellar production portfolio showcasing full-stack, mobile, and AI capabilities.
- Solve coding and algorithmic problems in live technical interview settings.

Grand Industry Capstone Project & Graduation Defense

SEM4 - CAP

Weeks 95–96 (48 Hours)

The ultimate 2-year capstone: Designing, building, testing, deploying, and...

CURRICULUM TOPICS & PRACTICAL LABS

Grand Capstone Deployment, Public Demo & Final Graduation Review

Tools: Cloud VPS Google Play Console Vercel GitHub

Full production launch, documentation package, mobile APK release, automated CI/CD pipeline, performance benchmark report, and presentation to industry judges.

KEY CONCEPTS: End-to-end software integration Production monitoring & reliability
Public technical documentation Final graduation thesis defense

 **Hands-on Lab Exercise:** Deliver the live keynote demonstration and code review defense of the Grand Capstone software suite.

✓ STUDENT COMPETENCIES & MEASURABLE OUTCOMES

- Graduate with an industry-grade, production-deployed enterprise software suite.
- Step into professional software engineering roles with complete confidence and proof of work.

Semester 4 Capstone Projects & Portfolio Proof of Work

GRAND CAPSTONE

Project 01

Grand Industry Capstone: 'NexusAI' Cross-Platform Intelligent Enterprise Operations Suite

An enterprise-grade cross-platform operations management platform featuring a high-performance Next.js 14 Web Portal, a Flutter Android Mobile App, an Express + PostgreSQL backend, real-time WebSocket communication, automated CI/CD pipelines, and an embedded Gemini AI Copilot providing natural-language data analytics, automated reporting, and intelligent task routing.

SYSTEM ARCHITECTURE:

Web Frontend: Next.js 14 (App Router) + TypeScript + Tailwind CSS

Mobile App: Flutter + Dart (Android & iOS with Push Notifications)

Backend API: Node.js + Express + Prisma ORM + PostgreSQL

Real-Time & Cache: Socket.io WebSockets + Redis Pub/Sub

AI Intelligence: Google Gemini 2.0 API with Function Calling & RAG

DevOps: Docker Compose, Nginx Reverse Proxy, SSL, Automated GitHub Actions CI/CD

REQUIRED DELIVERABLES:

- ✔ Live Public Web App URL with SSL & Custom Domain
- ✔ Signed Android Mobile APK release ready for device install
- ✔ Comprehensive Technical Documentation (OpenAPI Spec, ER Diagrams, Architecture Map)
- ✔ 100% Automated Test Suite (Unit, Integration, E2E via Playwright) in CI Pipeline
- ✔ 15-Minute Live Product Demonstration & Code Defense Presentation

Tools Next.js • Flutter • PostgreSQL • Docker • Gemini AI • GitHub Actions •
Mastered: Redis • Playwright

SEMESTER 4 WEEKLY ROADMAP & MILESTONE SCHEDULE

Timeline	Phase Name	Core Learning Focus	Deliverable Miles
Weeks 73–76	Phase 1: Architecture & SOLID	Clean Architecture, SOLID, Design Patterns, System Design	✔ Modular Enterprise
Weeks 77–80	Phase 2: Automated Testing & TDD	Vitest, React Testing Library, Supertest, Playwright	✔ 100% Covered
Weeks 81–85	Phase 3: Flutter Mobile Development	Flutter, Dart, Riverpod, Mobile APIs, Firebase Push	✔ Native Android
Weeks 86–89	Phase 4: Applied AI & Gemini SDK	Gemini API, Function Calling,	✔ Intelligent AI A

Timeline	Phase Name	Core Learning Focus	Deliverable Miles
		Prompting, RAG & pgvector	
Weeks 90–94	Phase 5: Agile, DSA & Career Launch	Jira Sprints, LeetCode DSA Patterns, Portfolio & Resumes	🕒 Polished Portfolio
Weeks 95–96	Phase 6: Grand Capstone Defense	Production deployment, APK builds, final graduation demo	🕒 Graduate as Job Ready